

基于 Socket 的通信仿真与 AI 译码验证平台设计

郭寒誠

摘要: 信道编码通过引入冗余信息，是保障数字通信可靠性的关键技术之一。近年来，基于深度展开的译码方法因将传统置信传播算法的迭代结构映射为可训练的前馈网络，兼具模型可解释性和数据驱动的性能优势。本文在 Linux 环境下采用 C 语言，从零设计并实现了一个基于 Socket 的分布式通信仿真与 AI 译码验证平台。平台以 BCH(63,36) 码为研究对象，由发送端、模拟信道和接收端三个独立进程构成，进程间通过 TCP 协议进行数据交互。发送端采用生产者-消费者线程模型，利用 POSIX 多线程机制并行生成信息比特并完成编码，将码字以字节压缩形式发送；模拟信道模拟加性高斯白噪声信道，对接收码字进行 BPSK 调制、叠加高斯噪声并计算对数似然比，最终将原始码字与软信息封装为通信帧转发；接收端则以纯 C 代码加载一个已在 Python 端训练完成的基于传统置信传播的深度神经网络译码器，严格复现变量节点更新、校验节点更新及 Sigmoid 激活的前向推理流程，并与作为标签的原始码字逐位比对，实时统计误码率。结果表明，该平台在 AWGN 信道下为 BCH(63,36) 码提供了不错的编码增益，经测试，深度神经网络译码器的误码率曲线在所有测试信噪比上均明显优于无编码 BPSK 理论曲线。本文工作为深度学习译码器在嵌入式 C 环境中的部署提供了一套完整的技术链路和可复用的验证工具。

关键词: 信道译码；深度展开；置信传播；Linux 系统编程

目 录

摘要	I
第一章 课题背景	1
1.1 引言	1
1.2 主要工作	1
第二章 关键理论与技术基础	3
2.1 BCH(63,36) 码	3
2.2 AWGN 信道模型	3
2.3 基于置信传播算法的深度学习神经网络译码器	4
2.3.1 置信传播算法	4
2.3.2 深度学习神经网络译码器	5
2.4 Linux 系统编程关键技术	5
2.4.1 POSIX 多线程编程	5
2.4.2 TCP Socket 网络编程	6
第三章 系统总体设计	7
3.1 系统总体架构	7
3.2 通信帧结构设计	8
3.3 发送端多线程设计	8
3.4 模拟信道设计	9
3.5 接收端译码流程	11
3.6 深度学习神经网络译码器的部署方案	11
第四章 平台实现	13
4.1 代码组织与编译管理	13
4.2 离线训练与模型导出	13
4.2.1 训练数据生成	14
4.2.2 网络构建	14
4.2.3 训练策略	15
4.2.4 模型导出	15
4.3 发送端实现	15
4.3.1 生产者—消费者模型	15

4.4	模拟信道实现	16
4.4.1	连接管理	16
4.4.2	AWGN 信道仿真	16
4.5	接收端实现	16
4.5.1	模型加载	16
4.5.2	深度神经网络译码器	16
4.5.3	Golden Test 验证	17
4.5.4	误码率统计	17
第五章	系统测试与结果分析	18
5.1	测试环境	18
5.2	功能验证	18
5.2.1	Golden Test 验证	18
5.2.2	模拟信道容错测试	18
5.3	译码性能测试	19
5.3.1	实验设置	19
5.3.2	实验结果	19
5.3.3	结果分析与讨论	21
5.4	开发中解决的关键工程问题	21
5.4.1	编码比特顺序不一致问题	21
5.4.2	离线训练模型 loss 反复震荡问题	21
5.4.3	多线程随机数竞争问题	22
5.4.4	模型部署一致性问题	22
第六章	展望	23
	参考文献	24

第一章 课题背景

1.1 引言

在数字通信系统中，信道编码是保障信息可靠传输的核心技术。通过在发送端引入受控冗余，信道编码使接收端能够在噪声干扰下检测并纠正传输错误，从而显著提升链路性能。自汉明码、BCH 码等经典分组码问世以来，纠错码理论已发展得相当成熟，并在深空通信、移动网络、光存储等领域得到广泛应用。然而，传统译码方法通常依赖于精确的信道统计模型，当模型不够准确或信道条件复杂时，其译码性能往往会出现退化。

近年来，深度学习在物理层通信中展现出引人注目的潜力。研究者开始将信道译码问题重新表述为一种学习任务：利用大量仿真数据，让神经网络自动学习从接收信号到发送码字的映射关系。在这条技术路线上，**深度展开**方法尤其值得关注。它并不把译码器当作一个黑箱网络，而是将传统置信传播算法 (BP 译码算法) 的迭代结构展开为一个前馈神经网络，每两层对应一次迭代，并在展开的边上引入可学习的权重参数。这些参数通过端到端训练获得，使得译码器既保留了传统算法的模型可解释性，又具备了数据驱动的性能优势。

Linux 操作系统是通信设备与嵌入式开发的主流平台之一，拥有成熟的网络协议栈和丰富的系统编程接口。利用 POSIX 多线程机制可以有效利用多核处理器并行生成仿真数据；利用 Socket 网络编程可以将发送端、信道与接收端拆分为独立的进程实体，以更贴近真实通信系统的方式构建仿真链路。这些系统级技术为验证 AI 译码器的可部署性提供了理想的支撑环境。

基于以上背景，本文选择 BCH(63,36) 码作为研究对象，在 Linux 平台上用 C 语言设计并实现了一个基于 Socket 的通信仿真与 AI 译码验证平台。系统由三个独立进程构成：发送端负责随机生成信息比特并进行 BCH 编码；模拟信道负责模拟 AWGN 信道并输出软信息；接收端则加载一个已在 Python 端训练完成的深度展开译码网络，以纯 C 代码完成在线推理，并实时统计误码率。通过该平台，本文不仅在 C 环境中验证了 BP 神经网络译码器的推理正确性和跨平台一致性，也为后续在真实嵌入式硬件上部署深度学习译码方案积累了一个可复用的实验工具。

1.2 主要工作

本文的主要工作可以概括为以下几个方面：

1. **自行搭建并离线训练深度神经网络译码器。**以 BCH(63,36) 码为对象，在 Python

环境中构建了一个 5 层的深度展开网络，最终获得了一个能够在 AWGN 信道下实现有效译码的深度神经网络译码器。

2. **设计并实现了一套完整的通信仿真平台。**平台采用多进程分布式架构，发送端、AWGN 模拟信道和接收端通过 TCP Socket 进行通信。
3. **将深度神经网络译码器部署到了纯 C 环境中。**将 Python 端训练好的深度展开网络导出为 C 语言头文件，在接收端用纯 C 代码实现了完整的前向推理流程。
4. **使用 Golden Test 验证机制。**在训练阶段生成固定测试向量及其参考输出，C 程序启动时自动执行一致性检验，确保权重数据加载无误、前向传播计算与训练环境完全一致。
5. **在 AWGN 信道下完成了译码性能评估。**测量了不同信噪比下深度展开译码器的误码率，验证了译码算法的有效性以及工程实现的正确性。

第二章 关键理论与技术基础

本平台旨在构建一个完整且可独立运行的通信仿真与 AI 译码验证环境。涉及技术包含 BCH(63, 36) 码, AGWN 信道模型, 传统置信传播译码算法, 基于置信传播算法的深度神经网络译码器, 同时充分融入 Socket 编程的关键技术。整体设计遵循“发送-传输-接收”的经典通信链路模型。

2.1 BCH(63,36) 码

BCH 码是一类由 Bose、Chaudhuri 和 Hocquenghem 于 1959-1960 年分别独立发现的能够纠正多个随机错误的循环码。BCH 码的构造基于有限域理论, 其核心优势在于可以根据目标纠错能力灵活设计码的参数, 在码率和纠错能力之间取得良好的平衡。

BCH 码可用生成矩阵 $\mathbf{G}$ 来描述。对于本系统所采用的 BCH(63,36) 码, 其基本参数为: 码长 $N = 63$ 比特, 信息位长度 $K = 36$ 比特, 校验位长度 $M = N - K = 27$ 比特。

BCH(63,36) 码采用系统码形式, 即码字的前 K 位 (或后 K 位) 直接包含原始信息比特, 其余位为通过生成矩阵计算得到的校验比特。

在本系统中, 编码通过生成矩阵 $\mathbf{G}_{K \times N}$ 实现。设信息比特向量为 $\mathbf{m} = [m_0, m_1, \dots, m_{K-1}]$, 则编码后的码字为 $\mathbf{c} = \mathbf{m} \cdot \mathbf{G} \pmod{2}$, 其中矩阵乘法在 $GF(2)$ 上进行 (即模 2 加法)。

2.2 AWGN 信道模型

加性高斯白噪声 (Additive White Gaussian Noise, AWGN) 信道是通信理论分析中最基础的信道模型之一。在该模型中, 接收信号是发送信号与统计独立的高斯噪声之和, 其数学模型为:

$$y_i = s_i + n_i \quad (2-1)$$

其中 s_i 为第 i 时刻的发送符号, $n_i \sim \mathcal{N}(0, \sigma^2)$ 为均值为 0、方差为 σ^2 的高斯噪声, y_i 为接收信号。

本系统采用二进制相移键控 (BPSK) 调制, 将码字比特映射为符号: 比特 0 映射为 +1, 比特 1 映射为 -1。发送符号的能量为 $E_s = 1$ 。信噪比 (Signal-to-Noise Ratio, SNR) 定义为每个信息比特的能量 E_b 与噪声功率谱密度 N_0 之比 E_b/N_0 , 通常以 dB 为单位。考虑到 BCH(63,36) 码的码率为 $R = 36/63$, 符号信噪比 E_s/N_0 与 E_b/N_0 的关系为:

$$\frac{E_s}{N_0} = R \cdot \frac{E_b}{N_0} \quad (2-2)$$

噪声方差 σ^2 与 E_s/N_0 的关系由下式给出：

$$\sigma^2 = \frac{1}{2 \cdot (E_s/N_0)} \quad (2-3)$$

在接收端，为了实现软判决译码，需要将接收符号 y_i 转换为对数似然比（Log-Likelihood Ratio, LLR）。在 BPSK 调制和 AWGN 信道下，LLR 的精确表达式为：

$$L_i = \log \frac{P(c_i = 0|y_i)}{P(c_i = 1|y_i)} = \frac{2y_i}{\sigma^2} \quad (2-4)$$

该 LLR 值作为软信息送入译码器，反映了每个接收比特为 0 或 1 的相对可能性。LLR 的符号指示硬判决结果（正数为 0，负数为 1），其幅值表示判决的置信度。

2.3 基于置信传播算法的深度神经网络译码器

2.3.1 置信传播算法

置信传播译码器（后称 BP 译码器）可以从 Tanner 图构建，Tanner 图是描述该码的某个校验矩阵的图形化表示。在该算法中，消息通过边传递。每条边根据其在所有边上接收到的所有输入消息（除了发送边上的消息）来计算其输出消息。

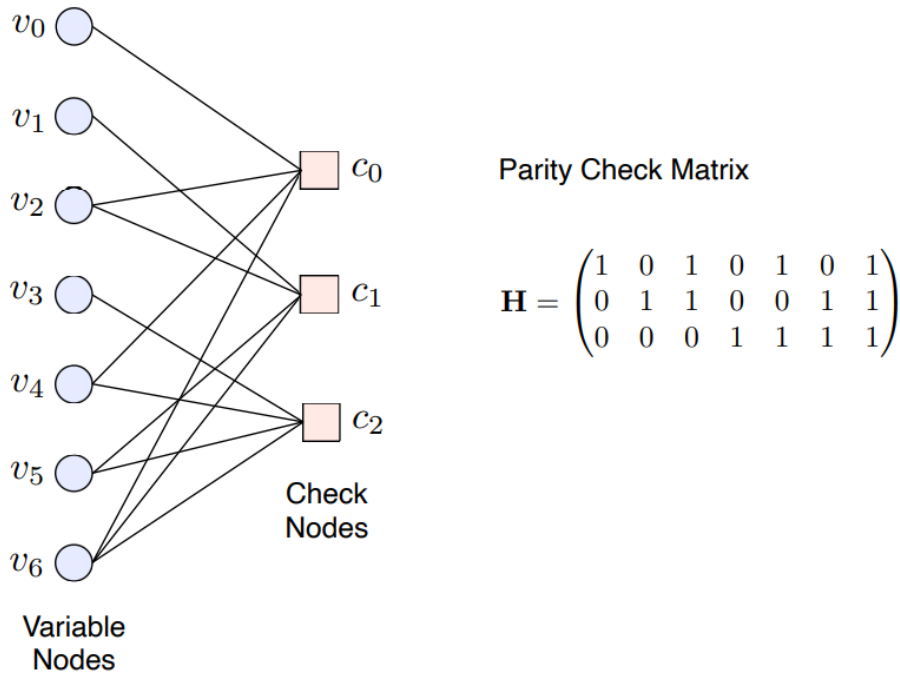


图 2-1 汉明 74 码的 Tanner 图示例

设有一 L 次完整迭代的 BP 算法，记 N 为变量节点的数目， M 为校验节点的数目， E 为边的数目，那么 BP 译码器的输入是信道输出的对数似然比组成的长度为 N 的向量。变量节点记为 v ($v = 1, 2, \dots, N$)，校验节点记为 c ($c = 1, 2, \dots, M$)，边记为

$e = (v, c)$ ($e = 1, 2, \dots, E$), 输入的 LLR 值由下式给出:

$$l_v = \log \frac{P(C_v = 0 | y_v)}{P(C_v = 1 | y_v)} \quad (2-5)$$

其中 y_v 表示对应第 v 个码处接收到的比特 C_v 。

2.3.2 深度神经网络译码器

通过深度展开的思想, 对 BP 算法的迭代结构进行展开, 每两层对应一次迭代, 每层的宽度为 E , 并在展开的边上引入可学习的权重参数。具体而言, 对于奇数层与偶数层会有不同的结构。对于奇数层第 i 层, 其方程为:

$$x_{i,e=(c,v)} = \tanh \left(\frac{1}{2} \left(w_{i,v} l_v + \sum_{e'=(v,c'), v' \neq v} w_{i,e,e'} \cdot x_{i-1,e'} \right) \right) \quad (2-6)$$

对于偶数层第 i 层, 其方程为:

$$x_{i,e=(c,v)} = 2 \tanh^{-1} \left(\prod_{e'=(v',c), v' \neq v} x_{i-1,e'} \right) \quad (2-7)$$

以及输出:

$$o_v = \sigma \left(w_{2L+1,v} l_v + \sum_{e'=(v,c')} w_{2L+1,v,e'} \cdot x_{2L,e'} \right) \quad (2-8)$$

其中 σ 表示 Sigmoid 激活函数。通过交叉熵损失函数进行训练, 优化所有权重参数。

该网络架构为非全连接神经网络, 在 [1] 中被提出, 时间复杂度与普通 BP 算法相似。

2.4 Linux 系统编程关键技术

Linux 操作系统为构建网络服务和高并发程序提供了丰富的系统编程接口。本平台中发送端的多线程并发、模拟信道的多连接管理以及通信, 均建立在这些关键技术之上。

2.4.1 POSIX 多线程编程

线程是操作系统调度的最小单位, 同一进程内的多个线程共享地址空间, 通信成本低、切换开销小。POSIX 线程标准为 Linux 平台提供了统一的多线程编程接口。本平台通过 `pthread_create` 函数创建线程, 每个线程独立执行指定的函数体, 通过 `pthread_join` 等待线程退出并回收资源。

多线程环境下, 多个线程并发访问共享数据时必须进行同步保护。互斥锁 (mutex)

用于保证同一时刻只有一个线程进入临界区，条件变量 (condition variable) 则允许线程在某一条件未满足时进入阻塞等待，从而避免忙等带来的 CPU 空转。本平台在发送端的共享队列上同时使用了互斥锁和两个条件变量，实现了高效的生产者 - 消费者模型。

2.4.2 TCP Socket 网络编程

Socket 是 Linux 下实现网络通信的标准编程接口。TCP 协议提供面向连接的、可靠的字节流传输服务，通过三次握手建立连接、四次挥手释放连接，保证数据按序到达且不丢失。本平台中三个进程之间的数据流转均基于 TCP Socket 实现。

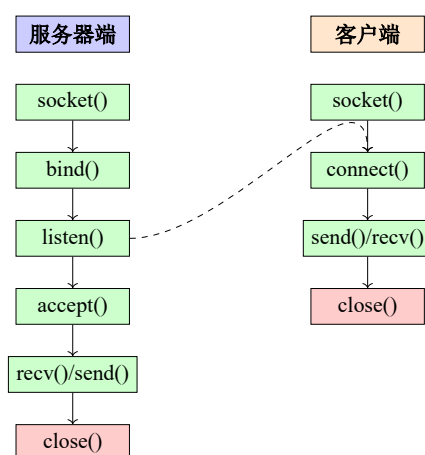


图 2-2 TCP Socket 通信流程

Socket 编程遵循客户端 - 服务器模型。服务端通过 `socket`、`bind`、`listen`、`accept` 调用序列在指定端口上等待连接请求；客户端通过 `socket`、`connect` 调用序列主动发起连接。连接建立后，双方使用 `send` 和 `recv` 函数进行双向数据传输。发送端和接收端均作为客户端主动连接信道模拟器，模拟信道则作为服务端同时监听两个端口。

第三章 系统总体设计

本章在前面关键理论与技术的基础上，从整体架构、通信帧格式、各进程的线程模型以及深度展开译码器的部署方案等方面，阐述系统的总体设计思路。全平台由三个独立进程构成，通过 TCP Socket 进行通信，形成一条完整的“发送—传输—接收”仿真链路。

3.1 系统总体架构

系统采用多进程分布式架构，三个进程分别为：发送端（tx）、模拟信道（channel）和接收端（rx）。三者均以 C 语言编写，运行于 Linux 环境，可部署于同一台或不同主机之上。它们之间的数据流与协议关系如图 3-1 所示。

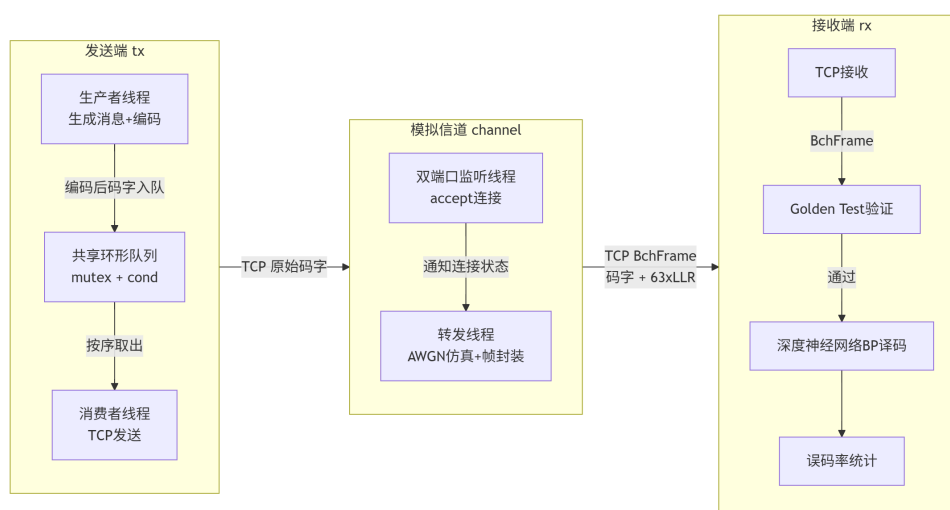


图 3-1 系统总体架构与数据流

发送端负责随机生成信息比特，完成 BCH(63,36) 编码，然后将码字以字节压缩形式（8 字节）通过 TCP 发送给模拟信道。模拟信道是系统中唯一永久在线的中间进程，它同时监听两个端口：一个等待发送端的连接，另一个等待接收端的连接。一旦收到发送端的原始码字，便模拟 AWGN 信道，对每一比特进行 BPSK 映射、叠加高斯噪声，然后计算对数似然比（LLR），最终将原始码字与 63 个浮点 LLR 封装成通信帧，转发给接收端。接收端以客户端模式主动连接模拟信道，对收到的每帧数据运行深度展开 BP 译码，并与原始码字对比统计误码率。

选择 TCP 作为传输层协议，主要是基于可靠性考虑。TCP 能够保证数据按序、无差错地到达对方，这对于需要精确统计误码率的仿真场景至关重要。同时，Linux 内核对 TCP 连接的优雅关闭支持（通过 `recv` 返回 0 感知对端断开）也为实现模拟信道的自动重连机制提供了便利。

3.2 通信帧结构设计

通信帧结构是连接三个进程的关键纽带，直接影响接收端进行误码率统计的便捷性与可靠性。本系统定义的通信帧结构体 `BchFrame` 包含两个字段：`codeword[8]` 和 `llr[63]`。前者存储 63 位原始码字的字节压缩形式，后者存储经过 AWGN 信道后得到的 63 个浮点 LLR 值。

这一设计的核心思路是将“标签”直接嵌入每一帧。原始码字在发送端产生后，由模拟信道原样携带至接收端。因为 BCH(63,36) 码是系统码，接收端在译码后需要与原始码字进行逐位比较，才能统计错误比特数。如果采用序号同步或独立传输标签的方案，需要额外设计同步协议或消耗带宽；而将原始码字与含噪信息打包在同一帧内，使得每一帧都自包含监督标签，接收端只需从帧中解出 `codeword`，即可零开销地获得译码对比的基准。

数据在三个进程中的格式变化如表 3-1 所示。

表 3-1 数据格式在各进程间的变化

进程	接收格式	发送格式
发送端 (tx)	—	8 字节原始码字
模拟信道 (channel)	8 字节原始码字	<code>BchFrame</code> (8 字节 + 63×4 字节)
接收端 (rx)	<code>BchFrame</code>	—

模拟信道从发送端收到的是 8 字节原始码字，解包为 63 比特后送入 AWGN 仿真模块，得到 63 个 LLR，然后将原始码字原样复制到 `BchFrame.codeword`，LLR 填入 `BchFrame.llr`，组成完整的帧发给接收端。接收端收到帧后，分别提取标签和译码输入，完成一次完整的仿真循环。

3.3 发送端多线程设计

发送端需要同时完成三项任务：随机消息生成、BCH 编码和网络发送。其中，消息生成与编码属于计算密集型操作，网络发送则属于 I/O 密集型操作。若采用单线程顺序执行，CPU 在等待网络 I/O 时将被闲置，导致整体吞吐量受限。因此，发送端采用“生产者—消费者”线程模型，将计算与 I/O 解耦，其结构如图 3-2 所示。

多个生产者线程并行运行，每个线程持有独立的随机数种子（通过 `rand_r` 保证线程安全），循环生成 36 位随机信息比特，利用预先加载的生成矩阵 **G** 完成 BCH 编码，得到 63 位码字，然后调用 `bits_to_bytes` 压缩为 8 字节，放入一个共享环形队列。

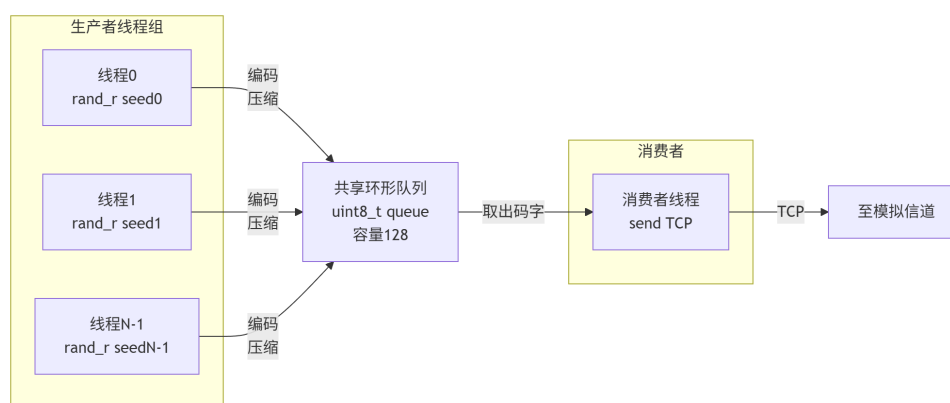


图 3-2 发送端多线程模型

共享队列采用固定长度数组配合头尾索引实现，由一把互斥锁和两个条件变量保护。当队列已满时，生产者线程阻塞在条件变量 `q_not_full` 上；当队列为空时，消费者线程阻塞在 `q_not_empty` 上。这种设计避免了忙等，同时保证了多生产者之间的互斥访问。

消费者线程从队列中按序取出压缩码字，通过程序启动时建立的一条 TCP 连接发送给模拟信道。

主线程注册了 `SIGINT` 和 `SIGTERM` 信号处理函数。收到退出信号后，全局标志 `keep_running` 被清零，并广播条件变量唤醒所有阻塞的线程。各线程检测到退出条件后依次安全退出，主线程通过 `pthread_join` 等待所有线程结束，最后关闭套接字并释放资源。

3.4 模拟信道设计

模拟信道是系统中连接发送端和接收端的核心中间件，需要同时管理来自两个方向的连接，并实时处理数据转发。为了提高响应性和隔离性，模拟信道内部采用多线程架构，将监听连接、接受连接、数据转发分别交给不同的线程执行。其状态机逻辑如图 3-3 所示。

两个独立的接受线程 `tx_acceptor` 和 `rx_acceptor` 分别在指定的端口上调用 `accept` 阻塞等待连接请求。一旦有对应端的连接到达，线程将已连接套接字存入全局变量，并通过条件变量通知转发线程。若当前已存在有效连接，新到达的连接将被直接拒绝并关闭，保证系统在任何时刻只维持一对一的收发关系。

转发线程 `forwarder` 是模拟信道的核心。它维护一个状态机，根据当前 `tx` 和 `rx` 的连接情况动态切换工作模式：

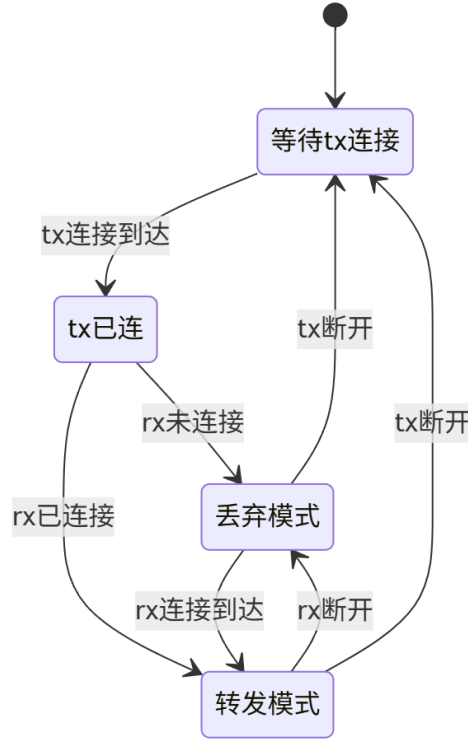


图 3-3 模拟信道转发线程状态机

1. **等待 tx 连接**：若 tx 尚未连接，转发线程阻塞等待条件变量 `tx_connected`，直到 tx 上线。
2. **tx 已连接，rx 未连接（丢弃模式）**：转发线程正常从 tx 读取 8 字节原始码字，解包并计算 AWGN 信道输出，生成完整的 `BchFrame`。但由于 rx 未上线，帧被直接丢弃，数据不被发送。这一设计保证了 tx 的发送缓冲区不会因无人接收而阻塞，发送端可以持续产生数据。
3. **tx 和 rx 均连接（转发模式）**：转发线程从 tx 收到码字后，解包、计算 LLR、组装 `BchFrame`，然后通过已建立的 rx 连接将帧发出。
4. **连接断开**：当 `send` 或 `recv` 返回错误时，转发线程主动关闭当前连接，重置对应全局套接字描述符为 -1，并自动回到相应的等待或丢弃状态，无需外部干预即可恢复。

AWGN 信道仿真的核心是生成高斯随机数。本系统采用 Box-Muller 方法，并接受一个线程本地的随机种子指针，确保在多线程环境下每个转发线程拥有独立且可复现的噪声序列。BPSK 映射和 LLR 计算遵循第二章给出的公式，SNR 由命令行参数指定。

3.5 接收端译码流程

接收端以客户端模式运行，启动后首先执行 **Golden Test**，通过验证固定测试向量的推理结果来确认模型数据加载无误、前向传播计算与 **Python** 训练环境一致。测试通过后，接收端主动连接模拟信道的 rx 监听端口，进入数据接收与译码统计的循环。

收到一个 **BchFrame** 后，接收端首先将 **codeword** 字段作为译码的标签。接着，将 **llr** 数组送入深度神经网络译码器。该译码器完全依据第二章所述的网络结构输出 63 个译码概率。

硬判决以 0.5 为阈值，将概率转化为 0/1 比特。然后逐位与标签比较，累加误比特数和总比特数，计算实时误码率。系统每收到 1000 帧输出一当前累计的 **BER**，收到退出信号后打印最终统计报告。

整个接收端的处理流程如图 3-4 所示。

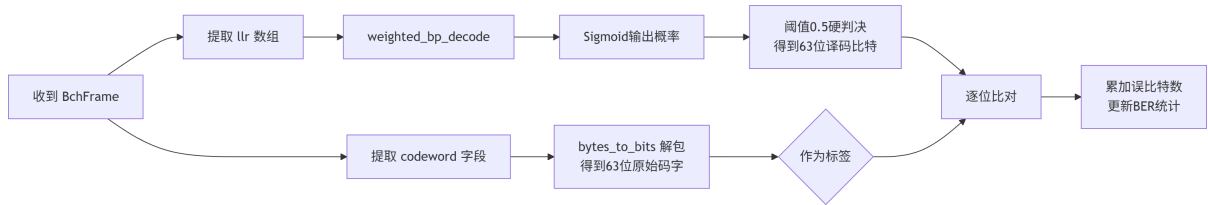


图 3-4 接收端单帧处理流程

3.6 深度神经网络译码器的部署方案

深度神经网络译码器在 **Python** 环境中完成训练后，需要将其全部结构信息和可学习参数部署到 **C** 语言接收端中。由于译码网络的连接模式由 **BCH(63,36)** 码的 **Tanner** 图决定，边索引、邻接表等结构数据与各层的权重矩阵是一一对应的，因此不能仅导出权重值，而必须将整个网络的结构和参数一并迁移。

具体部署流程如下：

1. **网络结构导出：**在 **Python** 端，根据校验矩阵 **H** 生成 **Tanner** 图。遍历所有变量节点和校验节点，记录每条边的索引、每个节点的邻居关系以及消息传递中的包含边集合。这些结构数据（包括 **bp_var_check_offsets**、**bp_check_var_data**、**bp_edge_idx**、**bp_inc_offsets**、**bp_inc_data** 等）被写入 **C** 语言头文件 **model.h**，以全局常量数组的形式存储。
2. **权重参数导出：**训练结束后，每层的输入缩放因子 $w_{i,v}$ 、边对权重 $w_{i,e,e'}$ 以及输出层参数 $w_{i,v,e'}$ 按展开层的顺序展平，与网络结构一起写入 **model.h**。

3. **C 端加载:** 接收端通过 `#include "model.h"` 直接获得所有常量数据, 无需运行时解析文件, 数据已编译为静态常量。
4. **Golden Test 验证:** 训练脚本同时生成一组固定的测试输入 (LLR 数组) 及其对应的参考输出概率, 存入 `golden_test.h`。接收端启动时, 用该组输入运行一次深度神经网络译码器, 将输出的 63 个概率值与参考值逐位比较。若最大误差小于预设阈值 (10^{-5}), 则证明权重加载和推理实现与 Python 端完全一致, 可以开始正常接收数据。

这一部署方案的核心优势在于: 通过网络结构与权重的联合编译, 避免了运行时解析带来的额外复杂度和出错可能; 同时, Golden Test 为跨平台模型部署的正确性提供了可量化的验证手段, 是确保从 Python 训练到 C 推理这一完整管线可靠性的关键环节。

第四章 平台实现

本章在系统总体设计的基础上，详细阐述各模块的具体实现方法。全平台由 C 语言编写的三个独立进程和 Python 离线训练脚本共同构成，本章依次介绍代码组织结构、离线训练与模型导出、发送端、模拟信道和接收端的实现细节。

4.1 代码组织与编译管理

项目根目录下分为 C 和 Python 两个子目录，分别存放系统源码与离线训练脚本。

Python 目录包含以下文件：

- **训练脚本 `train.py`**：基于 PyTorch 实现深度展开译码网络的构建、训练、测试，以及模型结构数据与权重的导出。
- **依赖清单 `requirements.txt`**：记录 Python 环境所需的外部库（PyTorch、NumPy、galois 等），便于复现训练环境。

C 目录包含全部 C 语言源文件、公共头文件、编译配置文件及构建输出：

- **公共头文件 `common.h`**：定义 BCH(63,36) 码的基本参数、通信帧结构体 `BchFrame` 以及比特与字节之间的转换工具宏。
- **发送端 `tx.c`**：负责多线程生成消息、BCH 编码并通过 TCP 发送压缩后的原始码字。
- **模拟信道 `channel.c`**：实现 AWGN 信道仿真、LLR 计算、通信帧封装与转发，支持动态连接管理。
- **接收端 `rx.c`**：加载深度展开译码网络的结构与权重，完成在线推理与误码率统计。
- **模型头文件 `model.h`**：由训练脚本自动生成，包含 Tanner 图结构、生成矩阵、校验矩阵以及所有可学习权重参数。
- **Golden Test 头文件 `golden_test.h`**：由训练脚本自动生成，包含一组固定测试输入及其参考输出概率。
- **编译配置文件 `CMakeLists.txt`**：管理三个可执行目标的构建依赖与链接库。
- **构建目录 `build`**：存放 CMake 生成的中间文件及最终可执行程序。

4.2 离线训练与模型导出

深度展开译码网络在 Python 环境中完成训练后，需要将全部结构信息和可学习参数导出为 C 语言头文件，供接收端加载。本节阐述训练数据生成、网络构建、训练策略以及导出流程。

4.2.1 训练数据生成

训练数据基于 BCH(63,36) 码的代数结构生成。利用 galois 库获取该码的生成矩阵 $\mathbf{G}$ 和校验矩阵 $\mathbf{H}$ 。

研究表明当在二进制记忆对称信道上传输时，误码率与传输的码字无关。因此，为了训练网络，使用由单个码字的带噪版本构建的数据集就足够了。为了方便，我们使用全零码字，它必然属于任何线性码。在每个训练批次中，将全零码字通过 AWGN 信道仿真模块得到对应的 LLR 向量，作为网络的输入。信道仿真采用与 C 端模拟信道完全一致的公式：BPSK 映射 ($0 \rightarrow +1$)、叠加高斯噪声、计算 $LLR = 2y/\sigma^2$ 。训练信噪比在 1 dB 至 6 dB 范围内随机选取，使网络能够学习到不同噪声强度下的译码策略。

测试阶段则使用随机生成的合法码字。具体方法为：随机生成 36 位信息比特，通过 $\mathbf{G}$ 矩阵在 GF(2) 上进行编码，得到 63 位码字，再送入 AWGN 信道生成 LLR 后评估译码性能。

4.2.2 网络构建

深度展开译码网络的构建分为两步：首先从校验矩阵 $\mathbf{H}$ 中提取 Tanner 图的结构信息，然后基于该结构实例化可训练的深度神经网络译码器模型 (WeightedBPDecoder)。

遍历 $\mathbf{H}$ 的所有元素，对每个值为 1 的位置 (i, j) ，记录一条从变量节点 j 到校验节点 i 的边，并为每条边分配唯一的整数索引。同时构建两组邻接表：var_to_check 记录每个变量节点连接的所有校验节点，check_to_var 记录每个校验节点连接的所有变量节点。

在这些邻接表的基础上进一步构建“边对”关系。对于每个变量节点 v ，遍历其所有邻居校验节点。对于每条出边 $e = (v, c)$ ，找出所有其他入边 $e' = (v, c')$ ($c' \neq c$)，将 (e, e') 记录为一个边对，并分配唯一的边对索引。这一关系对应于深度展开网络中奇数层的加权求和——每条输出边需要聚合来自所有其他输入边的消息。

WeightedBPDecoder 类继承自 PyTorch 的 Module，其可学习参数包括：

- **输入层权重** w_input：存储每层每个变量节点的输入缩放因子 $w_{i,v}$ 。
- **边对权重** edge_pair_weights：存储每层每对边的消息传递权重 $w_{i,e,e'}$ 。
- **输出层权重** w_out_node 和 w_out_edge：分别存储每个变量节点的输出缩放因子及每条关联边的输出权重。

所有可学习参数均以 $\mathcal{N}(0, 0.1^2)$ 随机初始化。网络迭代次数 num_iterations 设为 5，

即展开后的网络包含 5 个完整的“奇数层 + 偶数层”模块。

4.2.3 训练策略

训练使用 RMSprop 优化器，初始学习率 0.01，配合指数衰减调度器 ($\gamma = 0.95$) 逐轮降低学习率。损失函数为标准二元交叉熵，训练总轮数为 60 轮。

前向传播严格按第二章给出的公式执行。所有迭代层完成后，由输出层综合最终消息状态与输入 LLR，经 Sigmoid 产生 63 个比特的译码概率。

4.2.4 模型导出

训练完成后，调用 `export_to_c_header` 函数将全部网络结构与权重写入 `model.h`。该头文件以 C 语言全局常量数组的形式组织数据，包括：

- 基本参数宏定义 ($N = 63$ 、 $M = 27$ 、 $K = 36$ 、 E 、迭代次数、最大节点度数等)。
- 边索引矩阵 `bp_edge_idx`，维度 $N \times M$ ，记录每对变量-校验节点对应的边索引，无连接处填 -1。
- 生成矩阵 `bp_G` ($K \times N$) 和校验矩阵 `bp_H` ($M \times N$)，均为整型 0/1 数组，供发送端编码和后续可能的传统译码器使用。
- 四组权重数组：`bp_w_input`、`bp_edge_pair_weights`、`bp_w_out_node` 和 `bp_w_out_edge`，按展开层顺序以多维浮点数组存储。
- 四组 Tanner 图结构数组：`bp_var_check_data` 与 `bp_var_check_offsets` (变量节点邻接表)，`bp_check_var_data` 与 `bp_check_var_offsets` (校验节点邻接表)，`bp_inc_data` 与 `bp_inc_offsets` (边对包含关系表)，均采用偏移量 + 数据数组的紧凑存储方式。

同时调用 `generate_golden_test_header` 函数生成 `golden_test.h`。该文件包含一组用固定随机种子 (`seed=42`) 生成的测试输入 LLR 及其对应的网络输出概率。C 端接收端启动时用该组数据执行一致性检验，确保权重加载与推理实现完全正确。

4.3 发送端实现

4.3.1 生产者—消费者模型

多个生产者线程并行运行，每个线程持有独立的 `rand_r` 随机种子，循环生成 36 位随机信息比特。编码时直接利用 `model.h` 中导出的生成矩阵 `bp_G`，通过模 2 矩阵乘法得到 63 位码字。

编码完成后，调用 `common.h` 中的 `bits_to_bytes` 函数将 63 位码字压缩为 8 字节，

放入共享环形队列。队列采用固定长度数组配合头尾索引实现，由一把互斥锁和两个条件变量保护：队列满时生产者阻塞等待 `q_not_full`，队列空时消费者阻塞等待 `q_not_empty`。

消费者线程从队列中按序取出压缩码字，通过程序启动时建立的唯一 TCP 连接发送给模拟信道。

4.4 模拟信道实现

模拟信道是系统中唯一永久在线的中间进程，承担原始码字接收、AWGN 信道仿真、LLR 计算与通信帧封装转发的核心职责。其内部采用多线程架构，将监听连接、接受连接与数据转发分别交由不同线程处理。

4.4.1 连接管理

两个独立的接受线程 `tx_acceptor` 和 `rx_acceptor` 分别在指定端口上调用 `accept` 阻塞等待。一旦有对应端连接到达，线程将已连接套接字存入全局变量 `current_tx_fd` 或 `current_rx_fd`，并通过条件变量通知转发线程。若当前已存在有效连接，新到达的连接将被拒绝并关闭。

4.4.2 AWGN 信道仿真

AWGN 仿真的核心是高斯随机数生成。本系统采用 Box-Muller 方法，在 `gaussian_rand` 函数中实现。

BPSK 映射规则为 $0 \rightarrow +1$ 、 $1 \rightarrow -1$ ，叠加高斯噪声后，LLR 的精确计算公式为 $2y/\sigma^2$ ，其中 $\sigma^2 = 1/(2 \cdot 10^{\text{SNR}/10})$ 。SNR 由命令行参数指定。

4.5 接收端实现

接收端以客户端模式运行，主动连接模拟信道的 `rx` 监听端口。启动后首先执行 Golden Test，验证通过后进入数据接收与译码统计循环。

4.5.1 模型加载

接收端通过 `#include "model.h"` 直接获得深度展开译码网络的全部结构数据与权重常量。由于所有数据已编译为静态常量数组，避免了运行时解析文件可能引入的错误。

4.5.2 深度神经网络译码器

前向推理函数 `weighted_bp_decode` 严格复现 Python 端训练时的网络结构。输入为 63 个浮点 LLR 值，输出为 63 个译码概率。

推理流程按展开层的顺序执行。所有迭代层执行完毕后，输出层对每个变量节点计算加权和：输入 LLR 乘以 `bp_w_out_node`，加上所有关联边的消息乘以 `bp_w_out_edge`，经 Sigmoid 激活得到该比特为 1 的预测概率。

4.5.3 Golden Test 验证

Golden Test 在接收端启动时自动执行，是确保跨平台部署一致性的关键环节。训练脚本在导出 `golden_test.h` 时，用固定随机种子生成了一组 LLR 输入，并在 Python 端记录了网络的输出概率。接收端加载模型后，用同一组 LLR 调用 `weighted_bp_decode` 计算一次，将输出的 63 个概率值与 `golden_output` 逐个比较。若最大绝对误差小于 10^{-5} ，则打印验证通过信息并继续运行；否则输出警告。由于网络权重固定后是确定性函数，单一代表性样本即可证明全部参数加载无误、矩阵运算与激活函数实现完全正确。

4.5.4 误码率统计

Golden Test 通过后，接收端主动连接模拟信道，进入帧接收循环。每收到一个 `BchFrame`，先从 `codeword` 字段解包出 63 位原始码字作为标签，再将 `llr` 数组送入 `weighted_bp_decode` 得到译码概率。硬判决以 0.5 为阈值，将概率转化为 0/1 比特后与标签逐位比较，累加误比特数和总比特数。系统每收到 1000 帧输出一次当前累计误码率，收到退出信号后打印最终统计报告。

第五章 系统测试与结果分析

本章对平台进行功能验证与性能测试。首先说明测试环境，然后验证 Golden Test 一致性、多线程并发通信以及模拟信道的容错能力，接着重点分析深度展开译码器在 AWGN 信道下的误码率性能，最后对开发中解决的关键工程问题进行总结。

5.1 测试环境

所有测试均在以下环境中完成：

- **硬件平台：**x86_64 架构 laptop，AMD Ryzen7 5800H 处理器，16 GB 内存。
- **操作系统：**部署：Ubuntu 22.04 LTS、离线训练：Windows 11。
- **编译器：**GCC 13.2.0。
- **Python 环境：**Python 3.9, PyTorch 2.0.1, NumPy 1.21.6, galois 0.4.11, Scipy 1.13.1。
- **编译选项：**发送端与模拟信道链接 `-pthread`，接收端链接 `-lm`。

5.2 功能验证

5.2.1 Golden Test 验证

接收端启动后、连接模拟信道之前，自动使用 `golden_test.h` 中预存的测试向量执行一次前向推理，并将输出概率与 Python 端记录的参考值逐位比对。

```
==== Golden Test ====  
Max error: 0.000000e+00  
Golden test passed!  
=====
```

图 5-1 Golden Test 验证结果

测试结果如图 5-1 所示。C 端推理输出与 Python 参考值之间的最大绝对误差小于预设阈值 10^{-5} ，验证通过。该结果表明模型头文件 `model.h` 中的权重数据被完整且正确地加载，C 语言实现的变量节点更新、校验节点更新及 Sigmoid 激活逻辑与训练环境保持计算一致性。

5.2.2 模拟信道容错测试

模拟信道被设计为永久在线中间件，能够在发送端或接收端意外断开后自动恢复监听状态。测试中依次经历：仅发送端连接时的丢弃模式、接收端加入后的转发模式、接收端断开后恢复丢弃模式，以及发送端断开后重新等待连接。

图 5-2 的日志清晰记录了模拟信道的状态流转。整个过程中模拟信道无需人工重

```

ghc@ghc-OMEN:/media/ghc/F/DistributedCommSim/C/build$ ./channel 8000 8001 1.5
已转发 917000 帧
已转发 918000 帧
已转发 919000 帧
已转发 920000 帧
tx 连接关闭
转发线程: tx 连接已关闭, 等待新 tx 连接...
tx 已连接 (fd=8)
转发线程: 开始处理 tx (fd=8) 的数据
send to rx: Connection reset by peer
rx 连接断开, 恢复丢弃模式
tx 连接关闭
转发线程: tx 连接已关闭, 等待新 tx 连接...

```

图 5-2 模拟信道运行截图

启, 验证了动态连接管理与容错机制的有效性。

5.3 译码性能测试

5.3.1 实验设置

为评估深度展开译码器在 C 环境中的实际译码能力, 在不同信噪比下测量了系统的误码率。实验参数如表 5-1 所示。

表 5-1 译码性能测试参数

参数	取值
码字类型	BCH(63, 36)
信道模型	AWGN + BPSK
SNR 测试点	1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0 dB
每 SNR 测试码字数	100000
迭代次数 (网络层数)	5

每个 SNR 点测试 100000 个随机生成的合法码字。原始码字经模拟信道加噪并转换为 LLR 后, 送入接收端的 `weighted_bp_decode` 函数完成译码, 硬判决结果与原始码字逐位比对统计 BER。作为对比基准, 同时给出无编码 BPSK 在 AWGN 信道下的理论误码率曲线。

5.3.2 实验结果

表 5-2 列出了深度神经网络译码器在各 SNR 下的实测 BER。图 5-3 将实测数据与无编码 BPSK 理论曲线进行了对比。

从图 5-3 和表 5-2 可以清晰看出, 深度神经网络译码器的 BER 曲线在所有 SNR 点上均明显低于无编码 BPSK 的理论曲线, 且差距随 SNR 增加而迅速扩大。这表明

表 5-2 深度神经网络译码器 BER 实测数据

SNR (dB)	BER	Ratio
1.0	5.228857e-02	329418/6300000
2.0	2.772714e-02	174681/6300000
3.0	9.683175e-03	61004/6300000
4.0	2.024127e-03	12752/6300000
5.0	2.865079e-04	1805/6300000
6.0	3.666667e-05	231/6300000
7.0	5.396825e-06	34/6300000
8.0	6.349206e-07	4/6300000
9.0	4.761905e-07	3/6300000
10.0	0.000000e+00	0/6300000

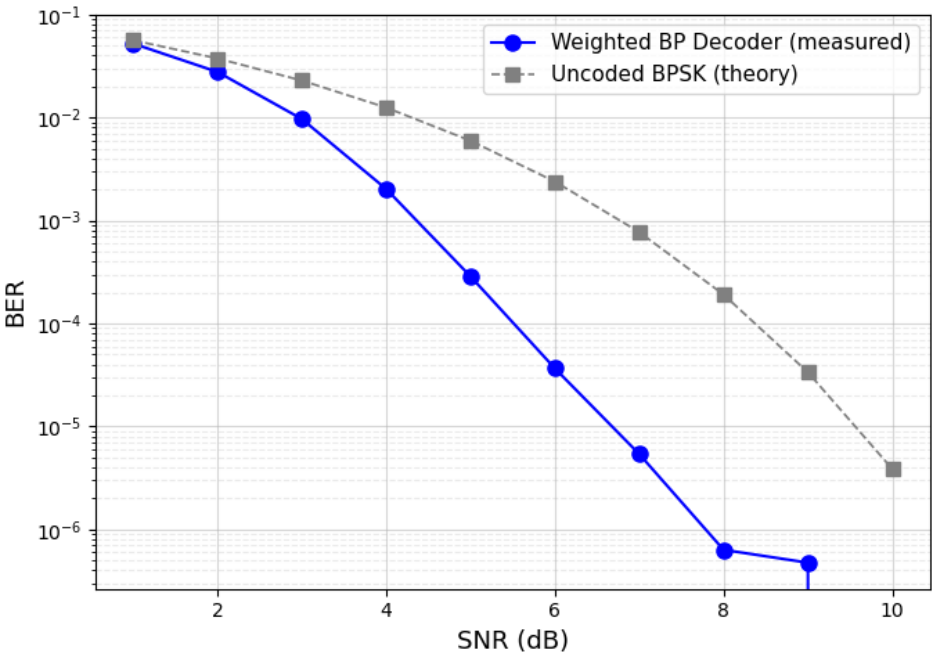


图 5-3 深度神经网络译码器 BER 性能曲线

BCH(63,36) 码的纠错能力得到了有效发挥，译码器向用户提供了显著的编码增益。在 SNR 为 7 dB 时，实测 BER 已几乎降至 0。这说明深度神经网络译码器在信道条件较好时可以近乎无误地恢复所有信息比特，验证了 5 次迭代网络对于 BCH(63,36) 码的充分性。

5.3.3 结果分析与讨论

上述实验结果验证了深度展开译码器在 C 环境中的有效部署。然而在实验过程中，也观察到训练过程对权重初始化存在一定敏感性。在相同的超参数设置下，仅部分随机种子能够收敛至上述具有明显编码增益的状态，而另一些种子则陷入译码性能与无编码水平相当的局部最优。

5.4 开发中解决的关键工程问题

在平台的开发与调试过程中，遇到并解决了若干跨平台部署与并发编程中的典型问题。

5.4.1 编码比特顺序不一致问题

项目初期并没有直接使用 BCH(63,36) 码和基于传统 BP 算法的深度神经网络译码器，而是出于验证目的简单实现了一个基于汉明 74 码以及简单神经网络的原型。在原型中，曾出现神经网络译码 BER 高达 0.24 而作为对比的硬判决译码 BER 仅为 0.02 的异常，这表明神经网络几乎完全没起作用。经排查，问题根源在于 Python 训练端与 C 推理端的码字比特排列顺序不一致，Python 训练端使用生成矩阵产生编码，而 C 端则因贪图简单是自行直接根据输入构造码字，未注意到 Python 端生成的码字比特顺序与 C 端不同，Python 端码字是在信息比特间的特定位置插入校验比特，而 C 端则是按照先信息比特而后校验比特的顺序构造输入。这种不一致导致网络学到的映射关系与实际输入完全错位。未注意到这一问题时，怀疑是神经网络部署出错，直接促成了后续阶段 Golden Test 验证机制的引入。

5.4.2 离线训练模型 loss 反复震荡问题

在训练深度神经网络译码器时，观察到 loss 曲线出现非常明显反复震荡的现象，无法稳定收敛，训练出的模型也效果不好。经多次排查，是由于训练初期，为了让模型在不同 SNR 条件下的泛化能力，在每个训练 epoch 内会在一个 SNR 范围内随机生成训练数据。例如在 SNR 范围 [1, 6] dB 内，每个 epoch 内的训练样本 batch 大小为 120，每个数据根据随机的 SNR 生成，这导致了模型可能某一次学习本身是在普遍较低 SNR 的数据下训练出来的 loss，因为较大的学习率导致更新方向的步幅过大，走到了一个较差的模型状态，但又因为下一次训练的数据是普遍较高 SNR 的数据，而高 SNR 数据通常比低 SNR 数据拥有更低的基准 loss，导致模型误以为上次更新的步幅是正确的，继续沿着这个方向更新，模型变得更差，loss 震荡，无法稳定收敛。

5.4.3 多线程随机数竞争问题

发送端最初使用 C 标准库的 `rand()` 函数生成随机比特。在多线程并发调用时，`rand()` 内部的全局状态被多个线程同时读写，产生数据竞争，各线程产生的序列高度相关甚至重复。将其替换为 POSIX 标准的 `rand_r()` 函数后，每个线程维护独立种子变量，彻底消除了竞争。同样，模拟信道中 Box-Muller 高斯噪声生成函数也改为接受线程本地种子参数，保证了噪声序列的独立性。

5.4.4 模型部署一致性问题

将深度学习模型从 Python 环境迁移至纯 C 环境，面临权重加载正确性和推理计算一致性的双重风险。为验证部署的正确性引入了 Golden Test 机制，能够在训练阶段即生成固定测试向量及其参考输出，C 端启动时自动执行一致性检验。为跨平台模型迁移的可靠性提供了方法论层面的保障。

第六章 展望

尽管本文已完成了一个功能完整的仿真验证平台，但受限于时间与实验条件，仍存在若干值得深入改进的方向：

1. **训练模型的优化。**在 Tanner 图的小循环结构中，信息通过某条边流入变量节点才是导致性能下降的根源。学术界已有研究证明，为每条边分配一个独立的权重，能获得大部分的性能增益。本平台部署的模型实现方式为每个“边对”一个独立权重，若以每条边分配一个独立权重的方式实现，算法复杂度会从 $O(LEd_{max})$ 降至 $O(LE)$ 。当节点最大度数较大时，节省的计算量会非常可观。
2. **扩展对更多码型和信道模型的支持。**本平台目前以 BCH(63,36) 码和 AWGN 信道为研究对象。将平台扩展至 LDPC 码、Polar 码等现代编码方案，以及瑞利衰落信道、干扰信道等更复杂的信道模型，将有助于进一步验证深度神经网络译码方法在不同通信场景下的适用性。
3. **嵌入式平台的实时性验证。**本文的测试均在 x86_64 架构的个人计算机上完成。将该平台移植至 ARM 嵌入式板卡（如树莓派或嵌入式 Linux 开发板），评估深度展开译码器在资源受限条件下的推理延迟和功耗表现，是检验其实际部署可行性的重要后续步骤。
4. **网络结构的轻量化研究。**当前深度展开网络包含 5 层迭代，每层包含与 Tanner 图边数相当的参数规模。探索网络剪枝、权重共享或蒸馏等轻量化方法，在保持译码性能的前提下降低推理计算量，对于该技术在实时通信系统中的应用具有重要意义。

参考文献

- [1] Eliya Nachmani, Yair Beery, and David Burshtein. Learning to decode linear codes using deep learning. *IEEE*, 2016.
- [2] John R Hershey, Jonathan Le Roux, and Felix Weninger. Deep unfolding: Model-based inspiration of novel deep architectures. *Computer Science*, 2014.
- [3] Toshiki Matsumine and Hideki Ochiai. Recent advances in deep learning for channel coding: A survey. 2024.
- [4] Nir Shlezinger, Jay Whang, Yonina C. Eldar, and Alexandros G. Dimakis. Model-based deep learning. 2020.
- [5] Shu Lin and D. Costello. Error control coding, second edition. *Prentice-Hall, Inc.*, 2004.
- [6] R. Gallager. Low-density parity-check codes. *IRE Transactions on Information Theory*, 8(1):21–28, 1962.
- [7] F. R. Kschischang, B. J. Frey, and H. A. Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.
- [8] J. G. Proakis and M. Salehi. Digital communications. *Digital Communications*, 73(11):3–5, 2015.
- [9] Stevens, W. Richard / Rago, and A Stephen. *Advanced Programming in the UNIX Environment*. Advanced Programming in the UNIX Environment, 2016.
- [10] 王远洋, 周渊平, and 郭焕丽. Linux 下基于 socket 多线程并发通信的实现. *微计算机信息*, (15):3, 2009.